

CodLab: Sensor Monitor Pro

Guía Maestra: MVVM, Kotlin 2.0 y Jetpack Compose

Proyecto de Ingeniería de Software · 2026

1 Introducción

Este laboratorio está pensado para estudiantes que se inician en el desarrollo Android moderno. Construirás, paso a paso, una aplicación que monitorea sensores aplicando los tres pilares profesionales:

- **Arquitectura MVVM:** separa la lógica de datos de la interfaz de usuario.
- **Programación Reactiva:** usa *StateFlow* para que la UI se actualice automáticamente.
- **Robustez de Datos:** aplica *Null Safety* para evitar cierres inesperados ante fallos de hardware.

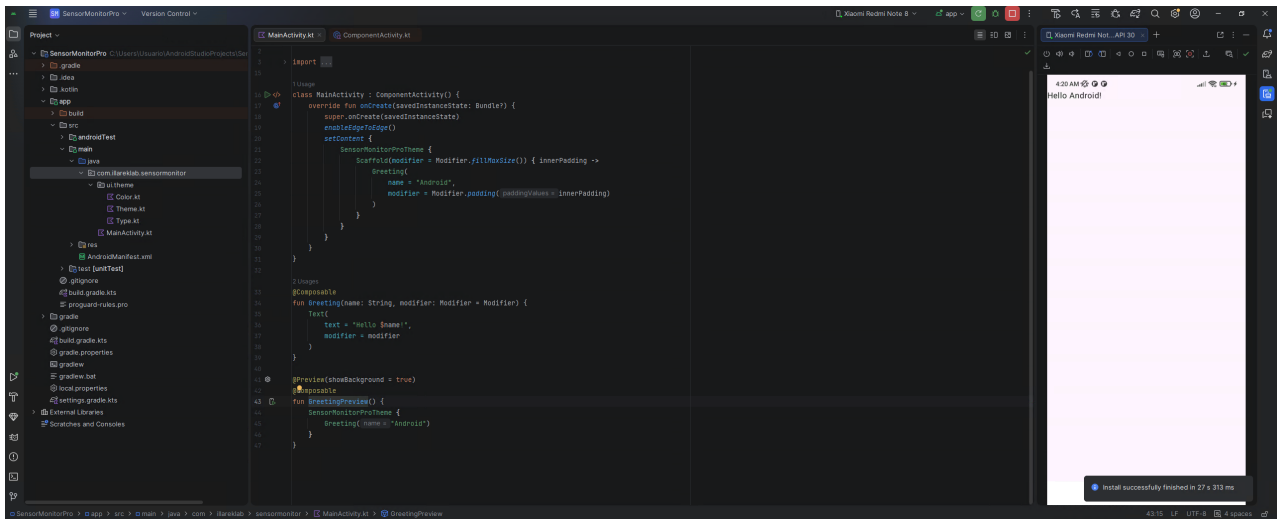
Antes de empezar

Necesitas tener instalado **Android Studio (Hedgehog 2024.1 o superior)**, con el JDK 17 incluido. No hace falta un teléfono físico: usaremos el **emulador integrado**.

2 Paso 1: Creación del Proyecto

1. Abre Android Studio y selecciona **New Project**.
2. Elige la plantilla **Empty Activity** (la que tiene el logo de Jetpack Compose, no confundir con *Empty Views Activity*).
3. Configura los parámetros del proyecto:
 - **Name:** SensorMonitorPro
 - **Package Name:** com.illareklab.sensormonitor
 - **Language:** Kotlin
 - **Minimum SDK:** API 24 (Android 7.0)
 - **Build Configuration:** Kotlin DSL (build.gradle.kts)
4. Pulsa **Finish** y espera a que termine el proceso de *Gradle Sync* (la barra inferior dejará de moverse).

Captura sugerida: ventana de configuración de *New Project* ya rellenada.



3 Paso 2: Estructura de Paquetes (Capas MVVM)

Android Studio no crea la estructura MVVM por ti. Tendrás que organizar el proyecto manualmente dentro de:

```
app > src > main > java > com.illareklab.sensormonitor
```

3.1 Crear los paquetes

1. Haz clic derecho sobre la carpeta **sensormonitor**.
2. Selecciona **New > Package**, escribe **model** y pulsa Enter.
3. Repite el proceso para crear el paquete **viewmodel**.

Sobre la carpeta ui

La carpeta `ui.theme` ya existe (la creó la plantilla). Tus archivos de pantalla (la **Vista**) los crearás **directamente dentro de ui**, no dentro de `ui.theme`. Esto evita conflictos de imports.

Estructura final esperada:

```
com.illareklab.sensormonitor
|-- model/
|-- viewmodel/
|-- ui/
|   |-- theme/ (ya existe)
|-- MainActivity.kt (ya existe)
```

Pegue aquí su captura del panel Project con las carpetas creadas

4 Paso 3: Capa Model (Datos)

Haz clic derecho en la carpeta **model** y selecciona **New > Kotlin Class/File > Class**. Nombra el archivo **SensorData**.

```
package com.illareklab.sensormonitor.model

/**
 * Data Class que representa la lectura de un sensor.
 * El campo "value" es Double? (anulable) para simular
 * sensores desconectados o con fallo de hardware.
 */
data class SensorData(
    val id: Int,
    val label: String,
    val value: Double?,
    val unit: String
)

// Funcion de extension con Operador Elvis (?:) para
// devolver "--" cuando el valor sea nulo.
fun SensorData.getReading(): String = "${value ?: "--"} $unit"
```

¿Qué es el operador Elvis?

`value ? : "--"` significa: *si value no es nulo, úsalo; si es nulo, usa "--*". Se llama así porque `?:` parece el flequillo de Elvis Presley.

5 Paso 4: Capa ViewModel (Lógica y Estado)

Crearemos dos archivos dentro de la carpeta **viewmodel**.

5.1 4.1 UIState.kt (Sealed Class)

Define los tres estados posibles que puede tener la pantalla.

```
package com.illareklab.sensormonitor.viewmodel

import com.illareklab.sensormonitor.model.SensorData

sealed class UIState {
    data object Loading : UIState()
    data class Success(val sensors: List<SensorData>) : UIState()
    data class Error(val message: String) : UIState()
}
```

5.2 4.2 SensorViewModel.kt

Es el cerebro de la aplicación. Carga los datos de forma asíncrona y los expone como un flujo inmutable.

```
package com.illareklab.sensormonitor.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelScope
import com.illareklab.sensormonitor.model.SensorData
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
```

```
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class SensorViewModel : ViewModel() {

    // Flujo MUTABLE: solo el ViewModel puede modificarlo (private)
    private val _uiState = MutableStateFlow<UIState>(UIState.Loading)

    // Flujo INMUTABLE: la UI solo puede leerlo
    val uiState: StateFlow<UIState> = _uiState.asStateFlow()

    init {
        viewModelScope.launch {
            delay(2000) // Simula la lectura de un hardware real
            _uiState.value = UIState.Success(listOf(
                SensorData(1, "Temperatura", 24.5, "C"),
                SensorData(2, "Humedad", null, "%") // Valor nulo de prueba
            ))
        }
    }
}
```

¿Por qué dos variables (_uiState y uiState)?

Es el patrón de **encapsulación**. La versión **private** se puede modificar dentro del ViewModel. La pública es de solo lectura, así garantizamos que la *Vista* no toque el estado por accidente.

6 Paso 5: Capa View (Interfaz Compose)

Esta es la pieza que convierte el estado en pantalla. Antes de escribir nada de código, hay que agregar una dependencia que no viene en el proyecto base.

6.1 5.1 Agregar la dependencia lifecycle-viewmodel-compose

La función `viewModel()` y el paquete `androidx.lifecycle.viewmodel.compose` no son parte de Compose ni de la librería base de ViewModel: viven en una librería puente aparte que tienes que añadir manualmente. **Si te saltas este paso, te aparecerán errores de *Unresolved reference 'compose'* y *Unresolved reference 'viewModel'* al compilar.**

1. En el panel **Project** de Android Studio abre el archivo `build.gradle.kts` (Module :app).

¡Cuidado! Hay dos archivos casi iguales

Existe `build.gradle.kts` (Project: SensorMonitorPro) y `build.gradle.kts` (Module :app). Tienes que abrir el segundo, el del **módulo** :app. Es el que contiene un bloque `android { ... }` y otro `dependencies { ... }`.

2. Busca el bloque `dependencies { ... }` (suele estar al final del archivo) y añade esta línea dentro:

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.10.0")
```

Te quedará algo similar a esto:

```
dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    // ... otras lineas que ya estaban ...
}
```

```
// LA LINEA NUEVA
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.10.0")
}
```

3. Aparecerá una barra amarilla en la parte superior con el botón **Sync Now**. Púlsalo y espera a que Gradle termine.

Pegue aquí su captura del bloque dependencies con la línea agregada

Versión moderna con *version catalog* (opcional)

Android Studio recomienda hoy declarar versiones en `gradle/libs.versions.toml`. Si quieres seguir ese estilo, añade en `[libraries]`:

```
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name =
"lifecycle-viewmodel-compose", version = "2.10.0" }
```

Y en `build.gradle.kts` usas: `implementation(libs.androidx.lifecycle.viewmodel.compose)`.

Para este laboratorio, el método del paso 2 (línea directa) es totalmente válido y más rápido para principiantes.

6.2 5.2 Crear SensorScreen.kt

Dentro de la carpeta `ui`, crea un nuevo archivo: clic derecho > New > Kotlin Class/File > File, nómbralo `SensorScreen`.

```
package com.illareklab.sensormonitor.ui

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.illareklab.sensormonitor.model.getReading
import com.illareklab.sensormonitor.viewmodel.SensorViewModel
import com.illareklab.sensormonitor.viewmodel.UISState

@Composable
fun SensorScreen(viewModel: SensorViewModel = viewModel()) {
    // Observamos el StateFlow y lo convertimos en estado de Compose
```

```

val state by viewModel.uiState.collectAsState()

Box(
    modifier = Modifier.fillMaxSize().padding(16.dp),
    contentAlignment = Alignment.Center
) {
    when (val s = state) {
        is UIState.Loading -> CircularProgressIndicator()

        is UIState.Error -> Text(
            text = "Error: ${s.message}",
            color = MaterialTheme.colorScheme.error
        )

        is UIState.Success -> LazyColumn(
            verticalArrangement = Arrangement.spacedBy(12.dp),
            modifier = Modifier.fillMaxWidth()
        ) {
            items(s.sensors) { sensor ->
                Card(modifier = Modifier.fillMaxWidth()) {
                    Column(modifier = Modifier.padding(16.dp)) {
                        Text(
                            text = sensor.label,
                            fontWeight = FontWeight.Bold,
                            style = MaterialTheme.typography.
                                titleMedium
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = sensor.getReading(),
                            style = MaterialTheme.typography.
                                headlineSmall
                        )
                    }
                }
            }
        }
    }
}

```

6.3 5.3 Conectar todo en MainActivity.kt

Abre el archivo MainActivity.kt (ya existe en la raíz del paquete) y reemplaza la llamada a `Greeting("Android")` dentro del bloque `Surface` por una llamada a tu nueva pantalla:

```

// Dentro de setContent { ... Surface { ... } }
SensorScreen()

```

No olvides el import:

```

import com.illareklab.sensormonitor.ui.SensorScreen

```

Recordatorio rápido si algo falla aquí

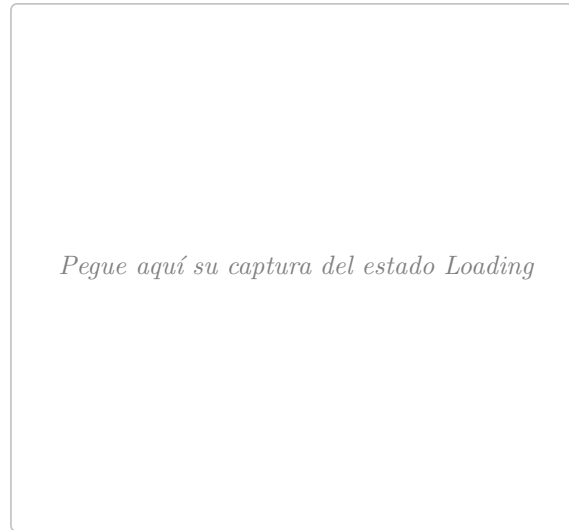
Si Android Studio marca `Unresolved reference "compose"` o `Unresolved reference "viewModel"`, regresa al **Paso 5.1**: te falta la dependencia `lifecycle-viewmodel-compose` o no pulsaste **Sync Now** después de añadirla.

7 Paso 6: Compilación y Resultados Esperados

Pulsa el botón verde **Run "app"** (o **Shift + F10**). Deberías observar dos estados consecutivos:

1. **Estado Loading:** durante los primeros 2 segundos aparece un círculo de carga centrado.
2. **Estado Success:** aparecen dos tarjetas:
 - **Temperatura:** 24.5 C
 - **Humedad:** --% (porque su valor es null)

Captura sugerida 1: estado de carga (los primeros 2 segundos).



Captura sugerida 2: estado final con los dos sensores.



8 Conclusión

¡Felicidades! Has construido tu primera aplicación con arquitectura MVVM real. Repasa lo que lograste:

- **Model:** clase `SensorData` con campos anulables y función de extensión.
- **ViewModel:** flujo de estado inmutable hacia la UI, mutable hacia adentro.

- **View:** Composable reactivo que reacciona automáticamente a los cambios.
- **Null Safety:** el operador Elvis (`?:`) protegió la app de un crash.

Reto opcional

Agrega un tercer sensor (*p. ej.* "Presión") y un botón **Refrescar** en la pantalla que vuelva a poner el estado en **Loading** y recargue los datos tras un **delay**. Pista: añade una función **refresh()** pública en el ViewModel.